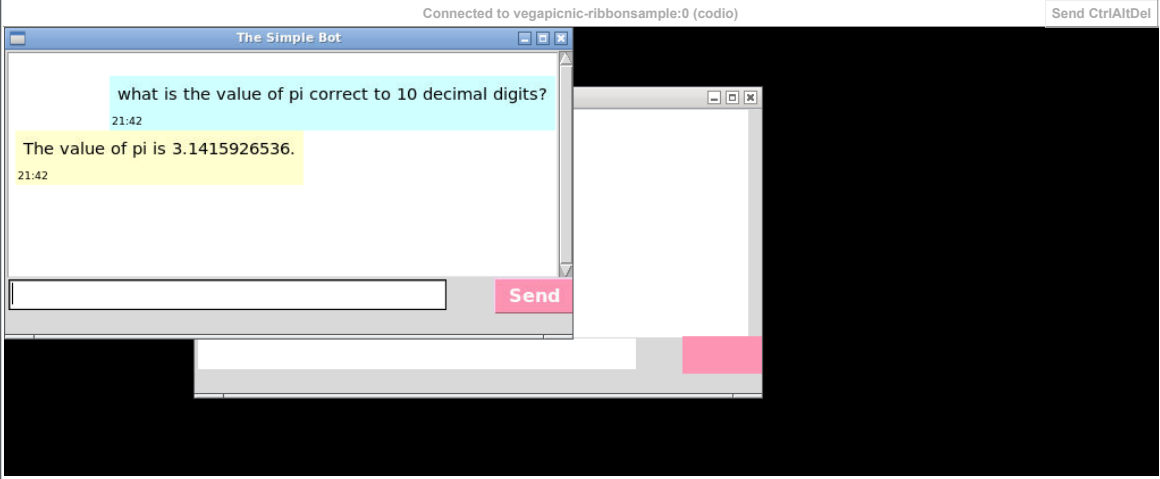


simplebot.py | terminal x
codio@vegapicnic-rtbbonsample:~/workspace\$ python3 simplebot.py



vegapicnic-rtbbonsample x simplebot.py
https://vegapicnic-rtbbonsample-3850.codio.io/

Codio x
Collapse A Simple Chatbot using NLTK

The bot response function using GPT-3

At this point, you must be familiar with OpenAI GPT-3 Toolkit. We are going to interact with our prompt to create a response. How we are going to interact with it is by changing our `get_bot_response` function.

Writing Code
To your left, you should see an opened file called `test.py`. Use this file as demo before we change our code.

Let's see how we can use `openai.Completion.create` to change our simple bot!

In the top left panel, the code file should have the same code as library as we are used to when interacting with GPT-3.

```
import os
import openai
openai.api_key=os.getenv('OPENAI_KEY')
```

Now we are going to create a function `get_bot_response`. The function `get_bot_response(user_input)` generates a bot response by sending a user's input to the OpenAI GPT-3 model, "text-davinci-002", and returns the model's generated text as the bot's response, stripped of leading and trailing whitespace.

```
# Generating response
def get_bot_response(user_input):
    prompt = f'Please provide a response to the following user input: {user_input}'
    response = openai.Completion.create(model="text-davinci-002",
                                        prompt=prompt,
                                        max_tokens=150,
                                        n=1,
                                        stop=None,
                                        temperature=0.5,
                                        )
    bot_response = response.choices[0].text.strip()
    return bot_response
```

TRY IT!

Command was successfully executed.

After making sure our code runs without errors we are ready to put it in our main code. Please make sure to copy everything in `test.py` and paste it after `from tkinter import *` in our `simplebot.py` (bottom left panel). Also make sure to delete our old `get_bot_response` function.

To run our code and try it, please click the button below. Use the other tab in the bottom-left panel

RUN CHATBOT

▼ Code
Your code should look like this:

```
import tkinter.scrolledtext as stx #creates a scrollable text window
import os
import openai

openai.api_key=os.getenv('OPENAI_KEY')

from datetime import datetime
from tkinter import *

# Generating response
def get_bot_response(user_input):
    prompt = f'Please provide a response to the following user input: {user_input}'
    response = openai.Completion.create(model="text-davinci-002",
                                        prompt=prompt,
                                        max_tokens=150,
                                        n=1,
                                        stop=None,
                                        temperature=0.5,
                                        )
    bot_response = response.choices[0].text.strip()
    return bot_response

def create_and_insert_user_frame(user_input):
    userFrame = Frame(chatWindow, bg="#d0ffff")
    Label(
        userFrame,
        text=user_input,
        font=("Arial", 11),
        bg="#d0ffff").grid(row=0, column=0, sticky="w", padx=5, pady=5)
    Label(
        userFrame,
        text=datetime.now().strftime("%H:%M"),
        font=("Arial", 7),
        bg="#d0ffff").grid(row=1, column=0, sticky="w")
    chatWindow.insert('end', '\n ', 'tag-right')
    chatWindow.window_create('end', window=userFrame)

def create_and_insert_bot_frame(bot_response):
    botFrame = Frame(chatWindow, bg="#ffffff")
    Label(
        botFrame,
        text=bot_response,
        font=("Arial", 11),
        bg="#ffffff",
        wraplength=400,
        justify="left").grid(row=0, column=0, sticky="w", padx=5, pady=5)
    Label(
        botFrame,
        text=datetime.now().strftime("%H:%M"),
        font=("Arial", 7),
        bg="#ffffff").grid(row=1, column=0, sticky="w")
    chatWindow.insert('end', '\n ', 'tag-left')
    chatWindow.window_create('end', window=botFrame)
    chatWindow.insert(END, "\n\n ")

def send(event):
    chatWindow.config(state=DISABLED)

    user_input = userEntryBox.get("1.0", 'end-2c')
    user_input_lc = user_input.lower()
    bot_response = get_bot_response(user_input_lc)

    create_and_insert_user_frame(user_input)
    create_and_insert_bot_frame(bot_response)

    chatWindow.config(state=DISABLED)
    userEntryBox.delete("1.0", 'end')
    chatWindow.see('end')

baseWindow = Tk()
baseWindow.title("The Simple Bot")
baseWindow.geometry("500x250")

chatWindow = stx.ScrolledText(baseWindow, font="Arial")
chatWindow.tag_configure('tag-left', justify='left')
chatWindow.tag_configure('tag-right', justify='right')
chatWindow.config(state=DISABLED)

sendButton = Button(
    baseWindow,
    font=("Verdana", 12, 'bold'),
    text="Send",
    bg="#f0d0b4",
    activebackground="#ff467e",
    fg="ffffff",
    command=send)
sendButton.bind("<Button-1>", send)
baseWindow.bind("<Return>", send)

userEntryBox = Text(baseWindow, bd=1, bg="white", width=38, font="Arial")

chatWindow.place(x=1, y=1, height=200, width=500)
userEntryBox.place(x=3, y=202, height=27)
sendButton.place(x=430, y=200)

baseWindow.mainloop()
```

And that's it! We've built our first chatbot using GPT-3! Let's run it and interact with it!

- How does the send function work in the given code?
- ☒ It displays the user's input and bot's response on the chat interface.
 - ☐ It takes the user's input and sends it to the GPT-3 model.
 - ☐ It opens and closes the chat interface.
 - ☐ It clears the chat interface.

The send function takes the user's input from the text entry box, generates a response from the bot using the `get_bot_response` function, and then calls `create_and_insert_user_frame` and `create_and_insert_bot_frame` to display the user's input and bot's response on the chat interface.

Check it!

Next